CRM Workflow Readiness and Critical-Path Estimation

PBL Milestone 1: Algorithm Architecture Specification and Software Engineering Proposal

III-BCSAD • bolanatics • 5 October 2026

Center for Research Management (CRM) Research Services and Information Management Platform

Repository: https://github.com/a-a123456789/umak-research-management

Member / student ID	ALGOCOM engineering role	Section responsibility
Balocating, Aaliyah Marie / A12448769	Requirements / technical lead	I and cross-section traceability
Maligaya, Gian Brent / A12448912	Algorithm designer	II and core implementation
Arnaez, Anjelo Benedict / K12254175	Complexity / benchmarking engineer	III and measured evidence
Bomediano, John David / A12448927	Architecture / repository engineer	IV and interface contract
Ducay, Justin Andrew / A12448793	Agile backlog / presentation lead	V–VI and defense deck

These are assigned responsibilities, not certification that each member personally performed the automated work. Each member must explain and review their section before submission. One designated member submits both the report and repository URL through TBL Hub. Deadline: confirm the course schedule.

Abstract and scope

CRM service processing involves prerequisites, document evidence, and distinct receipt and verification actions. This prototype identifies which incomplete steps are ready, explains blocked steps, and estimates the longest remaining dependency chain. It combines the supplied queue-based readiness implementation with DAG dynamic programming. The original repeated-scan algorithm and measurements are retained as a baseline. A separate Python worker is invoked by an authenticated Express endpoint; the mathematical core has no database or web-framework dependency.

The deliverable is a synthetic-data prototype and technical proposal. It does not implement a complete CRM frontend, persist service workflows, assign scarce staff, or predict calendar completion dates. Domain sequences must be confirmed against current university procedures before operational use. The research manual supplies business context rather than measured workloads [1].

I. Problem modeling

The supplied UMak manual defines the local service context [1]; task-graph scheduling literature supports modeling dependencies and explicitly warns that uncertain durations and execution conditions affect real performance [7]. Applying a DAG to CRM approval steps is this team's proposed model, not an empirical validation by the university. The cited study concerns distributed computational workflows; we borrow its dependency-model rationale, not its workload or measured results.

Domain interpretation

A service request is a workflow instance, distinct from a research record. Model steps such as receiving payment proof, staff verification, and processing a request separately. A received document is not automatically a verified document. The completed team drafts supply the workflow-readiness formulation; this revision retains it and adds a duration certificate to meet the advanced-paradigm requirement [2].

For example, let S4 be a completed prerequisite, S5 staff verification (15 estimated minutes), and S6 downstream processing (45 minutes), with S4 → S5 → S6. If payment proof has been received, a `received` gate on S5 is satisfied. S5 is ready; S6 remains blocked until S5 is actually completed. The structural remaining-duration estimate is 60 minutes, not a promised one-hour delivery. Nothing in the solver changes completion records.

Inputs, constraints and outputs

Let G=(V,E) be a directed graph with edge (u,v) meaning u must be completed before v starts. C is the set of completed steps. R(v) contains requirement/gate pairs for v. B is the collection of evidence records. A service adapter maps B to satisfied predicates S: verified evidence supplies both `id:verified` and `id:received`; received evidence supplies only `id:received`. Let d(v) be a nonnegative integer estimated duration in minutes.

Constraints: unique step IDs and edges; all endpoints and completed IDs exist; no directed cycle; C is prerequisite-closed; durations exist for every step; requirements use known steps; only received/verified gates are accepted. Historical evidence for already completed actions is not re-audited by the readiness core. Persistence and authorized evidence provenance are separate service-layer duties.

Readiness is defined exactly as Ready={v in V minus C : Pred(v) is a subset of C and R(v) is a subset of S}. Blocked steps report unmet prerequisites and missing evidence predicates. A valid all-completed graph returns COMPLETE. Invalid inputs, cyclic graphs, and inconsistent completion states return structured ERROR results.

The added objective is to compute T=max over directed paths P of sum of r(v) for v in P, where r(v)=0 for v in C and r(v)=d(v) otherwise. This is the minimum structural makespan under unlimited parallel capacity and precedence constraints only. The result includes all earliest structural finish values F(v), the maximum T, and an ordered critical-path certificate. For completed workflows T=0 and the certificate is empty. For zero-duration workflows several valid maximum paths can exist; deterministic input-order tie-breaking selects one.

Engineering limits: 5,000 steps, 25,000 edges, 50,000 requirement memberships, 50,000 evidence records, 2 MiB request body, IDs of 1–64 ASCII letters/digits/underscore/hyphen, and integer estimates of 0–525,600 minutes per step. These are chosen prototype bounds, not CRM demand measurements.

AI Assistance: ChatGPT/Codex assisted formalizing readiness, evidence gates, duration assumptions and the path certificate. The supplied sections also disclosed Claude assistance. Automated examples and equivalent-baseline checks support the formulation; team review remains required.

II. Paradigm selection, schema and pseudocode

Why DAG dynamic programming

Kahn traversal supplies a dependency order and detects cycles; dynamic programming evaluates overlapping prerequisite subproblems once in that order [3]. A join uses the maximum predecessor finish, not the sum, because independent predecessors may run concurrently. Greedy selection of the shortest available step cannot certify the longest prerequisite chain. Divide-and-conquer without memoization repeats shared ancestors. Exhaustive path enumeration can be exponential even though this DAG problem has a linear-time solution.

Schema: subproblem F(v) is the earliest structural finish of v; dependency subproblems are F(u) for u in Pred(v); recurrence is F(v)=r(v)+max({F(u):u in Pred(v)} union {0}); evaluation order is topological; certificate parent(v) stores one maximizing predecessor. Store predecessor choices rather than whole paths to avoid quadratic path copying. Use input rank to break equal finish ties in constant time per comparison.

Language-independent pseudocode

```
SOLVE(payload):
  validate fields, sizes, IDs, gates and nonnegative durations
  map receipt/verification evidence to satisfied predicates
  run supplied readiness checker: validate DAG and completed prefix
  if invalid: return its structured error
  construct predecessor lists, successor lists and indegrees
  enqueue every zero-indegree step
  while queue is not empty:
    v = dequeue()
    parent[v] = predecessor with largest F; ties by input rank
    F[v] = (F[parent[v]] if parent exists else 0) + r(v)
    for each successor w of v:
      indegree[w] -= 1
      if indegree[w] == 0: enqueue(w)
  endpoint = largest F; ties by input rank
  follow parent pointers backwards; omit completed prefix; reverse
  return readiness, F, F[endpoint], path and estimate limitations
```

The baseline retains repeated graph scans for validation and repeats scans to compute F only when all predecessors have already been computed. It uses the same recurrence, gates, durations and tie-breaking, making the comparison semantically equivalent. A baseline pass count records how strongly input order affects scanning.

Invariants and correctness

Indegree invariant: at every queue iteration the remaining indegree of an unprocessed vertex equals its number of unprocessed predecessors. Initially this holds by construction; processing v removes exactly its outgoing contributions. Therefore dequeued vertices have all prerequisites evaluated. A nonempty remaining graph with no zero-indegree vertex contains a directed cycle; the supplied validator reports it before DP.

DP induction: a source has no prerequisite delay, so F(v)=r(v) is correct. Assume predecessor finishes are correct. No valid execution can finish v before the latest predecessor plus r(v). Under unlimited parallel capacity all predecessor branches can finish at their computed values, so this lower bound is attainable. Thus the recurrence is exact for the stated structural model. The maximum finish gives the makespan, including disconnected components.

Certificate proof: every stored parent is an actual incoming neighbor achieving the maximum. Backtracking decreases topological rank, so it terminates without repetition. Summing r along the selected chain equals the endpoint finish by telescoping the recurrence. Prerequisite closure prevents a completed vertex after an incomplete ancestor; omitting the completed prefix preserves a valid remaining chain. Readiness soundness follows directly from both set-containment conditions; completeness follows because every incomplete vertex is examined. The algorithm never marks an action complete merely because it is ready.

AI Assistance: ChatGPT/Codex assisted the DAG recurrence, pseudocode, invariant and certificate reasoning, and regression tests. Claude assistance disclosed in the supplied algorithm section is retained. Human mathematical review has not been asserted.

III. Complexity and scalability

Let V count vertices, E count edges, Q count requirement memberships and B count evidence records. Treat bounded-size IDs and duration arithmetic as constant-size operations. Python dictionary/set membership has expected constant cost; this is not a claim of adversarial worst-case constant-time hashing [3].

Validation visits every supplied record: O(V+E+Q+B). Graph construction and the queue each cost O(V+E); scanning all predecessor lists in the DP costs O(E); backtracking costs O(V). Therefore valid proposed processing is O(V+E+Q+B), with linear best, typical and structural worst cases for full valid input under the hashing assumption. Malformed input may fail sooner. No input-order distribution is needed for the queue bound.

The baseline may need O(V) passes, each costing up to O(V+E), plus O(Q+B) evidence processing: O(V(V+E)+Q+B). Ordered DAGs can finish scans in one pass and attain O(V+E+Q+B). Reversed chains need V dependency passes, giving quadratic chain behavior. Average behavior depends on ordering and graph-depth distributions; no universal average Theta claim is made.

Auxiliary storage includes adjacency, predecessors, indegrees, queue, finish/parent maps, normalized requirements and evidence: O(V+E+Q+B). Output additionally holds readiness reasons and finish/path data, bounded by O(V+E+Q). Keeping input and output separately does not make the total memory O(V) when the graph and requirements are also materialized. The baseline has the same asymptotic storage bound; faster does not necessarily mean smaller peak allocation.

Dimension	CRM concern	Prototype response and limit
Volume	Large graphs or many evidence memberships	Linear core; V/E/Q/body caps; actual counts require institutional discovery
Velocity	Bursts of solve requests	Two concurrent workers per API process, no queue, timeout and 503 backpressure; no production throughput claim
Variety	Missing fields, cycles, receipt/verification distinctions	Strict schema, structured errors, normalized gates; documents and rich text excluded from core

Measured experiment

Original readiness-only measurements remain in `algocom/results.csv` and `results.md`; they must not be presented as timings for the extended solver. New results are `algocom/extended_results.csv` and `extended_results.json`. Inputs use seed 20261001 and hash seed 0. Each method receives the same synthetic workflow; canonical readiness and exact duration/certificate equality are checked before timing. Nine cases cross 10/100/1,000 steps with shuffled sequences, branching DAGs, and reversed sequences. Durations use a separate deterministic seed.

One untimed warm-up precedes ten timed repetitions. Garbage collection occurs outside each interval; medians use `time.perf_counter` [4]. Peak Python allocations use a separate `tracemalloc` run [5], excluding prepared input storage. Fixed method order, one seed and one machine limit inference; memory is not process RSS. Timings demonstrate measured behavior, not a proof of complexity.

Workflow / V	E	Proposed ms	Baseline ms	Speedup	Peak KiB P/B	DP scans B
sequence / 10	9	0.0368	0.0385	1.05	10.8/10.1	4
sequence / 100	99	0.2324	0.7972	3.43	82.3/84.4	55
sequence / 1000	999	2.6785	63.7561	23.8	779.1/778.4	502
branch / 10	16	0.0341	0.0361	1.06	9.3/8.5	3
branch / 100	156	0.2953	0.4597	1.56	91.6/94.1	16
branch / 1000	1718	3.5737	21.4473	6.0	846.6/845.3	141

Workflow / V	E	Proposed ms	Baseline ms	Speedup	Peak KiB P/B	DP scans B
sequence_reversed / 10	9	0.0297	0.0395	1.33	9.6/8.8	10
sequence_reversed / 100	99	0.225	1.2558	5.58	82.6/84.9	100
sequence_reversed / 1000	999	2.6634	113.4524	42.6	776.2/775.5	1000

Local environment: CPython 3.14.4, macOS 26.6.2 arm64, 10 logical CPUs. The separate CI performance budget uses three 5,000-step synthetic families and ten warmed repetitions each. Nearest-rank p95 is the maximum of ten observations. Local p95 values were 14.8234 ms (sequence), 19.8240 ms (branch), and 14.2090 ms (reversed), below 1,000 ms. The budget is a regression guard; GitHub's Linux/Python 3.12 runner is a different environment.

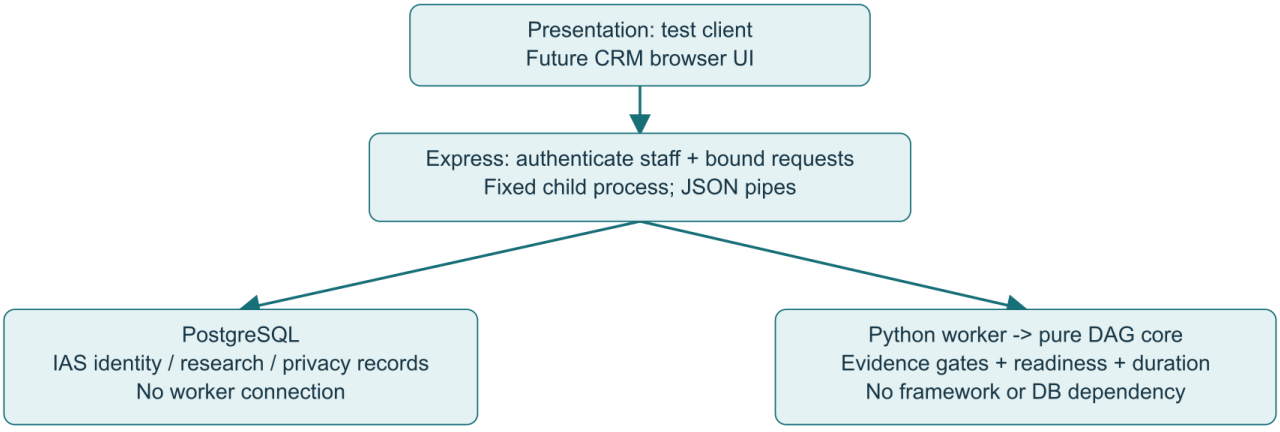
AI Assistance: ChatGPT/Codex assisted complexity accounting, benchmark extension and arithmetic checks. Actual timings came from executed scripts; team review of the assumptions remains required.

IV. Decoupled software architecture

The prototype has four responsibilities. Presentation is a future CRM UI; tests currently act as the client. Express authenticates and authorizes staff, parses a bounded scenario, and manages worker lifetime. Python normalizes gates and computes the pure result. PostgreSQL stores existing IAS identities/research/privacy records; the worker has no connection to it. A future workflow repository adapter must fetch authorized evidence instead of trusting arbitrary student assertions [6].

```
Client / test harness (future browser UI)
  -> HTTP + bearer -> Express authentication and staff authorization
  -> bounded orchestration -> fixed Python child via JSON stdin
  <- result + worker metrics <- pure ALGOCOM core via JSON stdout
Express -> PostgreSQL: identities, sessions, research and privacy records
Python worker -- no database dependency or database credentials
```

Algorithm component architecture



Responses return over the same interfaces. Limits: 2 concurrent children, 5 seconds, 2 MiB input.

The child uses no shell and receives only PATH, locale and hash-seed environment settings. Admission is capped at two concurrent children per API process. Timeout is five seconds; disconnect kills the child; output over 8 MiB fails. These are admission and lifecycle controls, not a container/OS memory sandbox. No asynchronous queue, Redis or production service deployment is claimed.

Exact public request

Machine-readable JSON Schema Draft 2020-12 contracts are supplied under `algocom/schemas/` [8]. Structural schemas supplement, rather than replace, Python checks for graph references, cycles, aggregate memberships and completed-state consistency. Express validates the full worker response, including types, input membership and COMPLETE invariants, before returning it. Malformed results produce 502 WORKER_PROTOCOL; valid existing payloads retain their behavior.

`POST /api/v1/algorithm/solve`, JSON body, staff bearer authorization. Exactly six fields are required; no additional top-level fields are accepted. `steps` and `completed_actions` are ID arrays. `dependencies` is an array of two-ID arrays. `requirements` maps step IDs to arrays of exactly `{id,gate}`. `evidence` is an array of exactly `{id,state}`. `durations` maps every step to its integer estimate. Section I states all bounds.

```
{"steps":["S4","S5","S6"],
 "dependencies":[["S4","S5"],["S5","S6"]],
 "completed_actions":["S4"],
 "requirements":{"S5":[{"id":"payment_proof","gate":"received"}]},
 "evidence":[{"id":"payment_proof","state":"received"}],
 "durations":{"S4":10,"S5":15,"S6":45}}
```

Example response (elapsed metric varies; 0.42 is illustrative):

```
{"status":"SUCCESS","ready_steps":["S5"],"blocked_steps":[{"step":"S6","reasons":["Unsatisfied prerequisite: S
5"]}],"earliest_finish_minutes":{"S4":0,"S5":15,"S6":60},"remaining_duration_minutes":60,"critical_path":["S5"
,"S6"],"duration_model":"Estimated dependency-only duration; unlimited parallel capacity; evidence and
resource delays excluded","metrics":{"worker_elapsed_ms":0.42}}
```

The success response carries readiness status and reasons, step-to-finish map, integer remaining duration, critical path, limitations, and `metrics.worker_elapsed_ms`. The elapsed metric covers Python-side parsing/solve, not total HTTP latency. Readiness reason ordering is not part of the semantic contract.

HTTP errors: 400 invalid JSON/schema/cycle/state, 401 unauthenticated, 403 non-staff, 413 excessive body, 503 worker capacity/unavailability, 504 timeout, 502 bad exit/protocol/output. Validation errors use `{status:"ERROR",error_type,message}`. Worker lifecycle errors also use the structured status/error_type/message envelope; parser and authorization errors use the API's `{error}` object. Cancellation after disconnect has no deliverable HTTP response. Exact examples and assertions are in `test/algorithm.test.js` and `docs/ALGORITHM_API.md`.

AI Assistance: ChatGPT/Codex assisted the process boundary, API contract and failure handling. The supplied architecture section disclosed Claude assistance. Automated API/worker checks do not substitute for institutional architecture approval.

V. Agile backlog and benchmark assertions

Story / ticket	Owner	Acceptance and evidence	Current status
US-A1: As CRM staff, I want ready and blocked steps with reasons so I can identify the next permissible action. ALG-01	Aaliyah / Gian	Receipt enables its receipt gate only; verification gate remains blocked. Cycles and inconsistent completed prefixes rejected. Core equals baseline on 1,500 randomized comparisons across retained and extended suites.	Implemented and automated checks pass; human domain review pending
US-A2: As a service planner, I want a critical-path estimate so I can identify the dependency bottleneck without promising a calendar deadline. ALG-02	Gian / Anjelo	Hand-computed chain, fork/join, disconnected graph, ties, all-complete and zero-duration assertions; certificate sum equals reported maximum; nine benchmark equivalence checks.	Implemented; duration inputs remain synthetic estimates
ALG-03: Isolate authenticated solve orchestration	John David	401/403, schema/body errors, direct-worker equality, timeout, capacity, cancellation and invalid worker result tests. No core DB imports.	Implemented locally and CI
ALG-04: Reproducible performance guard	Anjelo	Fixed seed, ten warmed observations, separate memory measurement, p95 <1,000 ms for three 5,000-step families.	Budget passed locally and integration CI
ALG-05: Defend and confirm domain semantics	Justin / all	Each member explains recurrence, trace, bounds and one failure mode; institution confirms gates/durations.	Human/institutional work remains

Run `python3 -m unittest discover -s algocom -p 'test_*.py' -v`, then `PYTHONHASHSEED=0 python3 algocom/bench_extended.py`. The extended runner fails on inequivalent results or a missed budget. GitHub runs correctness and a separate warmed performance guard; committed local result files are not silently replaced by CI measurements. Review workload conditions before changing a budget; never remove correctness assertions to make a benchmark pass.

Engineering implementation tickets

Ticket / module / algorithm	Numbered engineering steps	Edge cases and executable acceptance
ALG-01 / engine.py + prototype.py / readiness	1. Validate graph and state. 2. Normalize received/verified gates. 3. Evaluate unchanged readiness core.	Empty/cyclic graphs, unknown IDs, duplicate evidence and incomplete prerequisites; `python3 -m unittest discover -s algocom -p 'test_*.py' -v`
ALG-02 / engine.py / DAG dynamic programming	1. Build adjacency/indegree. 2. Relax longest prerequisite finish. 3. Reconstruct maximizing parent certificate.	Chain, fork/join, ties, completed and zero durations; test_engine.py asserts known results and 500 baseline comparisons
ALG-03 / src/algorithm.js + algorithm-result.js / bounded orchestration	1. Authenticate staff. 2. Bound input and children. 3. Validate output. 4. Kill failed/timed-out jobs and release slots.	Malformed output, wrong types, nonzero exit, 8 MiB output, cancellation and timeout; `npm test` requires 401/403/413/502 and valid-result agreement
ALG-04 / bench_extended.py / comparative profiling	1. Generate fixed-seed inputs. 2. Assert equality. 3. Time ten warmed runs. 4. Profile memory separately.	Nine cases; `performance_budget()` asserts each family p95 <1,000 ms at V=5,000; performance excludes memory profiling
ALG-05 / report + team review / domain validation	1. Confirm requirement gates and durations with CRM. 2. Rehearse certificate and errors. 3. Record genuine review.	External human acceptance; no automated assertion can prove understanding or institutional approval

VI. Academic defense answers

1. Why decouple heavy computation from the host framework and ORM?

A pure function can be tested against explicit inputs without a web request or database. A child process keeps CPU-heavy Python work outside the Node event loop, allows timeouts, and avoids passing database credentials into the solver. It adds process-start/serialization overhead, which is acceptable for this bounded prototype but should be measured before scaling. Decoupling alone does not stop resource exhaustion; concurrency limits are also necessary. [3], [6].

2. How would you mitigate exponential worst-case behavior?

This DAG longest-path formulation is polynomial, so exponential path enumeration is unnecessary. If a future problem adds scarce resources or combinatorial assignment constraints, identify that new problem's hardness, cap input size, impose time budgets, use branch-and-bound or approximation where justified, and report incomplete/approximate results honestly. Do not transfer the current linear proof to a harder scheduling problem. [3].

3. Why exact computation rather than a heuristic?

The constrained DAG objective has an efficient exact recurrence. A heuristic could miss the true bottleneck without providing a benefit at these limits. The answer is exact relative to supplied durations and edges, while the durations themselves remain estimates. Exact algorithmic computation is not accurate calendar prediction. [3].

4. Why use empirical benchmarks as well as asymptotic analysis?

The proof predicts growth independent of one machine. Measurements expose constant factors, Python allocation, input-order effects and operational regressions. A speedup greater than one favors the proposed method in that row only. Neither a fast run nor nine cases proves correctness or a universal speed advantage. Equivalent outputs must be established before timing. [3]–[5], [7].

5. How do you handle ML cold start or sparse data?

No ML model or recommendation engine is implemented, so cold start is not a current solver issue. The prototype uses explicitly provided duration estimates. If future learning estimates durations, start with transparent staff-reviewed defaults, display uncertainty, distinguish missing data from zero duration, and evaluate on held-out historical requests without leakage. Keep the dependency solver deterministic and separate from prediction. [7].

6. How do you prevent edge cases from crashing the host?

Reject malformed fields and resource-limit violations; detect cycles and inconsistent completed states; compute iteratively to avoid recursion depth failures; return structured errors; constrain child concurrency and lifetime; handle startup failure, bad JSON output and disconnect cancellation. Tests exercise those paths and confirm the host can process subsequent work. OS-level memory isolation and distributed admission control remain production follow-ups. [3], [6], [8].

Verification, provenance and handoff

Code evidence commit: `655114e0f4edf16697b6d4a9ebc9979573af91e0`. Local evidence records 19 Python tests and 11 backend test groups passing; randomized cases are inside those groups. GitHub integration runs 37184676703 and 37184676704 passed. The subsequent public-search cap adds a twelfth backend test group, which passed locally; final-main CI status is recorded separately in the repository completion ledger.

The supplied completed Tasks I–IV and original prototype/tests were retained as source material. Corrections clarify receipt/verification, add DAG DP, include requirement/evidence costs in bounds, remove unsupported deployment claims, and replace hypothetical benchmark numbers with executed outputs. Original source documents remain unchanged. All team members must still review, understand and rehearse the work; no signatures or institutional validation are manufactured.

References

[1] University of Makati, The University of Makati Research Manual, supplied institutional PDF, revised 2025. Business context only; current operational applicability requires confirmation.

[2] bolanatics, ALGOCOM Milestone 1 Task Guide, supplied completed team draft, 2026; and course PBL Milestone 1 Specification, supplied assignment document, 2026. Unpublished course materials.

[3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. MIT Press, 2022. https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/

[4] Python Software Foundation, “time — Time access and conversions,” Python 3.12 documentation, accessed Oct. 4, 2026. https://docs.python.org/3.12/library/time.html

[5] Python Software Foundation, “tracemalloc — Trace memory allocations,” Python 3.12 documentation, accessed Oct. 4, 2026. https://docs.python.org/3.12/library/tracemalloc.html

[6] OpenJS Foundation, “Child process,” Node.js v24 documentation, accessed Oct. 4, 2026. https://nodejs.org/docs/latest-v24.x/api/child_process.html

[7] J. Beránek, S. Böhm, and V. Cima, “Analysis of Workflow Schedulers in Simulated Distributed Environments,” arXiv:2204.07211, 2022. https://doi.org/10.48550/arXiv.2204.07211

[8] A. Wright et al., JSON Schema Validation: A Vocabulary for Structural Validation of JSON, Draft 2020-12, 2022 revision. https://json-schema.org/draft/2020-12/json-schema-validation

The presentation deck follows these references and contains exactly 14 slides. Documentation access dates do not replace publication dates; original foundational concepts are attributed to the 2022 textbook edition.

CRM workflow readiness and critical path

Anjelo Benedict Arnaez · K12254175	Complexity / benchmarking
Justin Andrew Ducay · A12448793	Backlog / presentation
Aaliyah Marie Balocating · A12448769	Requirements / technical lead
Gian Brent Maligaya · A12448912	Algorithm designer
John David Bomediano · A12448927	Architecture

III-BCSAD · bolanatics · 5 October 2026

Operational problem and limits

01 Determine the next permissible research-service action from prerequisites and evidence.

02 Estimate the dependency bottleneck using DAG dynamic programming and supplied durations.

03 No guaranteed calendar date: staff capacity and missing-evidence delays are excluded.

[1], [3], [7]. Proposed CRM model; institutional gates/durations need confirmation.

Receipt is not verification

01 A received payment proof can enable staff verification.

02 It does not complete that action or satisfy a verified gate.

03 Service workflows are distinct from stored research records.

[1], [2]

A precise graph model

01 Inputs: DAG G=(V,E), completed C, requirements R, evidence B, durations d.

02 Ready(v): every prerequisite completed AND every evidence gate satisfied.

03 Output: readiness reasons, earliest finishes, longest duration and path.

[3]

The objective is structural duration

01 r(v)=0 for completed steps; otherwise r(v)=d(v).

02 F(v)=r(v)+max predecessor F; an empty maximum is zero.

03 T=max F(v): unlimited parallel capacity; staffing and evidence waits excluded.

[3]

Why dynamic programming fits

01 Evaluate shared prerequisite subproblems once in topological order.

02 Joins take the maximum finish, not the sum of parallel branches.

03 Keep one maximizing parent to recover a path certificate.

[3]

Walk through the certificate

01 S4 completed → S5 verification (15 min) → S6 processing (45 min).

02 Received proof makes S5 ready; S6 is blocked until S5 completes.

03 F={0,15,60}; critical path [S5,S6]; estimate 60 min.

Repository example.json

Correctness depends on invariants

01 Queue indegree equals the count of unprocessed prerequisites.

02 Induction: every predecessor finish is final before a step is evaluated.

03 Backtracking actual maximizing edges certifies the reported longest path.

[3]

Growth is linear in the full input

01 Proposed: O(V+E+Q+B) under expected constant-time hashing.

02 Repeated scans: O(V(V+E)+Q+B) upper bound; ordered cases can be linear.

03 Storage includes graph, normalized evidence and output—not just the queue.

[3]

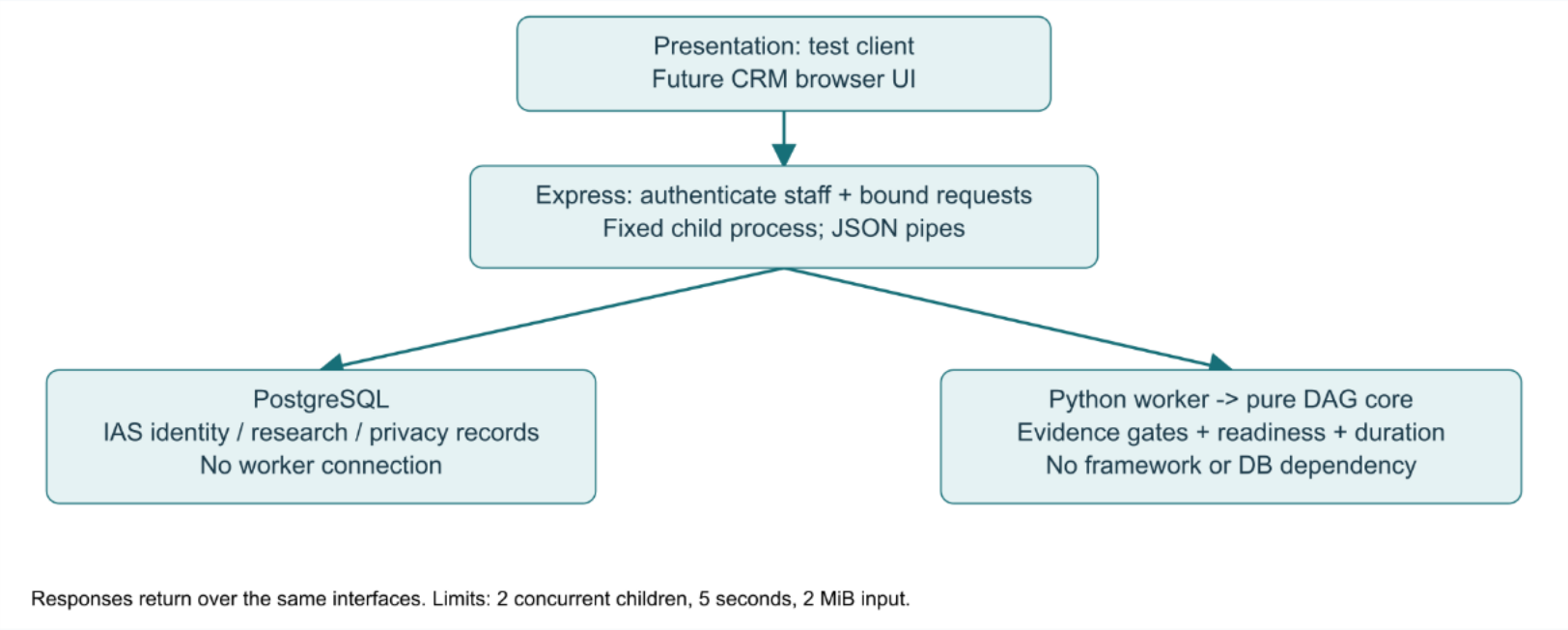
Measured comparison: 1,000-step cases



Median milliseconds; ten warmed repetitions; same synthetic input per pair.
Timing and memory were measured separately. Speedups are machine/input dependent.

[4], [5]; extended_results.json

Decoupled client, API, database and worker



Staff authentication precedes 2 MiB parsing. Python receives no database credentials.

Complete worker-result validation; 2 jobs / 5 s / 8 MiB output.

[6], [8]; machine-readable schemas under algocom/schemas/.

Tickets make the claims testable

01 US-A1 / ALG-01: explain ready and blocked steps; receipt/verification tests.

02 US-A2 / ALG-02: prove hand-computed durations and valid certificates.

03 ALG-03/04: API failure tests; 5,000-step warmed p95 budget <1,000 ms.

Tests and CI; Q cap 50,000 memberships

Defense: exact model, estimated inputs

01 Why decouple? Testability, bounded lifecycle and event-loop isolation.

02 Why exact? This DAG objective is polynomial; harder resource scheduling is different.

03 Benchmarks complement proof; ML cold start is not a current feature.

Full six answers in report VI; review edge-case handling

References and evidence

[1] University of Makati, Research Manual, revised 2025. Operational applicability needs confirmation.

[2] bolanatics, ALGOCOM Task Guide and PBL Milestone 1 Specification, 2026. Course materials.

[3] Cormen et al., Introduction to Algorithms, 4th ed. MIT Press, 2022.

[4] Python Software Foundation, Python 3.12: time, accessed 2026.

[5] Python Software Foundation, Python 3.12: tracemalloc, accessed 2026.

[6] OpenJS Foundation, Node.js v24: Child process, accessed 2026.

[7] Beránek, Böhm and Cima, “Analysis of Workflow Schedulers in Simulated Distributed Environments,” 2022.

[8] Wright et al., JSON Schema Validation, Draft 2020-12, 2022 revision.

Synthetic results are machine/order dependent. AI assistance disclosed in report I–IV; human review remains required.